CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 09
Minimum Spanning Trees, DSU & Structural Graph Reasoning

Two structural rules explain every MST algorithm. A data structure with an almost-constant amortized cost makes one of them fast.

Saturday 21 November 2026 · 3 hours

PS3 released today (due Sat 05 Dec)

Motivating problem: two networks

A · Fibre between cities

$n = 2 \times 10^5$ sites, $m = 6 \times 10^5$ candidate links along roads, each with a cost. Connect everything at minimum total cost.

Sparse: $m/n = 3$.

B · Clustering sensor readings

$n = 10^4$ points; any two can be linked, with a cost equal to their distance. There are $\binom{n}{2} = 5 \times 10^7$ edges, *not stored*: computed on demand.

Dense: $m/n \approx 5000$.

DISCUSS · 3 MIN

Would you use the same MST algorithm for A and B? What does each algorithm need to **store**, and what does it need to **sort**?

BLOCK A · 00:10–01:00

MST structure and algorithms

Cut and cycle properties · Kruskal · Prim · Borůvka

The cut property

$G = (V, E)$ connected, undirected, weights w. A **cut** $(S, V \setminus S)$ is **crossed** by an edge with one endpoint on each side. It **respects** an edge set A if no edge of A crosses it.

THEOREM (CUT PROPERTY, GENERIC FORM)

Let A be contained in some MST, let the cut $(S, V \setminus S)$ respect A, and let e be a lightest edge crossing it. Then $A \cup \{e\}$ is contained in some MST.

PROOF (EXCHANGE)

Let $T \supseteq A$ be an MST. If $e \in T$, done. Otherwise $T + e$ contains a cycle through $e = (u, v)$. The T-path from u to v crosses the cut at some edge e'. $e' \notin A$, since the cut respects A. And $w(e) \leq w(e')$, since e is a lightest crossing edge. $T' = T + e - e'$ is a spanning tree with $w(T') \leq w(T)$, so it is an MST containing $A \cup \{e\}$. ■

The cycle property and the blue/red framework

> **CYCLE PROPERTY**
>
> If e is the **unique** heaviest edge on some cycle of G, then e belongs to no MST. (Proof: an exchange argument, part of PS3.)

Tarjan's two rules (colour the edges; uncoloured = undecided):

- **Blue rule:** pick a cut with no blue edge crossing it; colour a lightest uncoloured crossing edge blue.
- **Red rule:** pick a cycle with no red edge; colour a heaviest uncoloured edge on it red.

Theorem: applying the rules in *any* order until all edges are coloured leaves the blue edges forming an MST.

Algorithm	Which rule, which cut
Kruskal	blue: the cut (component of u, rest) for the lightest edge between two components
Prim	blue: the cut (tree so far, rest)
Borůvka	blue, for every component at once
Reverse-delete	red: delete the heaviest edges that are not bridges

Kruskal and Prim: implementation decides cost

Kruskal

```
sort edges by weight
for each edge (u,v) in order
    if FIND(u) ≠ FIND(v):  add (u,v); UNION(u,v)
```

Sort $O(m \log m) = O(m \log n)$ + DSU $O(m\,\alpha(n))$. If the edges arrive pre-sorted (or are small integers: radix sort): nearly linear.

Prim

```
grow a tree T from any start vertex
repeat n − 1 times: add the lightest edge leaving T
```

Prim's priority queue	Time
unsorted array (adjacency matrix)	$\Theta(n^2)$
binary heap, lazy (push on each relaxation)	$O(m \log n)$
binary heap with decrease-key	$O(m \log n)$
Fibonacci heap	$O(m + n \log n)$ (amortized)

LEGACY CORRECTION

The old slides quoted "$O(E + V \log V)$ with a priority queue (heap)" for a lazy **binary-heap** implementation. That code is $O(E \log V)$. $O(E + V \log V)$ requires a Fibonacci heap. Always name the priority queue.

Borůvka (1926): all components at once

```
F = n singleton components
while F has more than one component
    for each component C
        find the lightest edge leaving C
    add all these edges to F    ▷ merge components
```

- Each phase at least **halves** the number of components, so there are $\leq \log_2 n$ phases.
- Each phase is $O(m)$, so the total is $O(m \log n)$.
- Naturally **parallel and distributed**: components act independently.

TIE-BREAKING IS ESSENTIAL

With equal weights, components may pick edges that form a **cycle**: in a triangle of weight-1 edges, each vertex may pick a different edge. Break ties consistently (e.g. by edge index) to make the weights effectively distinct.

THEORY

Borůvka phases + contraction are the basis of the randomized expected linear-time MST algorithm (Karger, Klein, Tarjan 1995). The best deterministic bound is $O(m\,\alpha(m, n))$ (Chazelle 2000).

Exercise A: choose, justify, adapt

1. Choose an MST algorithm (and priority queue, if any) for: (a) network A from the motivating slide; (b) network B; (c) a graph distributed over 64 machines; (d) edges arriving already sorted by weight from a database.
2. In a triangle whose edges all have weight 1, show how Borůvka without tie-breaking can produce a cycle. Give a tie-break rule and argue that it is consistent.
3. Some edges are **required** (they must be in the tree) and some are **forbidden**. How do you adapt Kruskal? When is the problem infeasible?
4. How do you compute a **maximum** spanning tree? Does the cut property need to change?

Pairs · 10 min

BLOCK B · 01:10–02:00

Disjoint-set union

Linking rules · path compression · $O(m\,\alpha(n))$ amortized

The DSU abstract data type

Maintain a partition of $\{1..n\}$ under

- `FIND(x)`: the representative of x's set;
- `UNION(x,y)`: merge the two sets.

Forest representation: each set is a rooted tree, the root is the representative, and `FIND` follows parent pointers.

Naive linking (root of x under root of y, always): $n-1$ unions can build a path, and each `FIND` on its bottom costs $\Theta(n)$.

USES

- Kruskal's cycle test
- Connectivity under edge insertions (online)
- Unification in type inference and compilers
- Image segmentation / connected components of pixels
- The scheduling matroid of L08: the "latest free slot"
- Offline LCA (Tarjan)

Not supported: splitting a set (deletions). Dynamic connectivity with deletions is much harder.

Union by size: logarithmic depth

Rule: link the root of the *smaller* tree (fewer nodes) under the root of the larger.

LEMMA

With union by size (and no path compression), every node has depth $\leq \log_2 n$. So `FIND` is $O(\log n)$ worst case.

PROOF

A node's depth increases (by exactly 1) only when its tree is linked under another root. That happens only if its tree was the smaller one, so the size of the tree containing the node at least **doubles**. Sizes never exceed n, so this happens at most $\log_2 n$ times.

■

UNION BY RANK

Store an upper bound on the height (*rank*) instead of the size, and link the lower rank under the higher; increment on ties. Same $O(\log n)$ guarantee. The rank analysis is PS3 Q3.

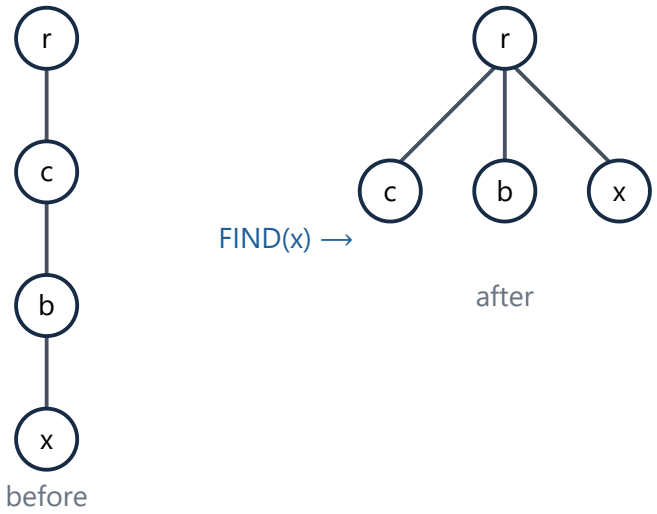
Why ranks? They stay meaningful under path compression (heights do not), and they are the quantity used in the $\alpha(n)$ analysis.

Path compression

```
FIND(x)
  if parent(x) ≠ x
     parent(x) = FIND(parent(x))
  return parent(x)
```

After a `FIND`, every node on the path points directly to the root.

- **Compression alone** (arbitrary linking): $O(\log n)$ *amortized* per operation (Tarjan & van Leeuwen 1984). A *single* FIND can still cost $\Theta(n)$.
- An iterative two-pass version avoids recursion depth; *path halving* and *path splitting* are one-pass alternatives with the same bounds.



LEGACY CORRECTION

The old slides stated "path compression: $O(\log n)$ in the worst case". Path compression gives an **amortized** bound. One operation can be linear.

Both heuristics: $O(m\,\alpha(n))$ amortized

THEOREM (TARJAN 1975; TARJAN & VAN LEEUWEN 1984)

With union by rank (or size) and path compression, any sequence of m operations on n elements takes $O(m\,\alpha(n))$ time, where α is the inverse Ackermann function. This is **tight**: $\Omega(m\,\alpha(n))$ holds for any pointer-based DSU (Tarjan 1979) and in the cell-probe model (Fredman & Saks 1989).

Ackermann-type function: $A_0(j) = j + 1$, and $A_k(j) = A_{k-1}^{(j+1)}(j)$ (iterate $j + 1$ times).

$A_1(1) = 3,\ A_2(1) = 7,\ A_3(1) = 2047,\ A_4(1) \gg 10^{80}$.

$\alpha(n) = \min\{k : A_k(1) \geq n\}$, so $\alpha(n) \leq 4$ for every n that could ever occur.

READ THE THEOREM CAREFULLY

- It is **amortized**: a total over the sequence, not a bound on each operation. A single FIND can still take $\Theta(\log n)$.
- "$\alpha(n) \leq 4$ in practice" is a statement about α, not about the constant hidden in $O(\cdot)$.
- Legacy slides: "$\alpha(n)$ … reaching 4*n at most" is meaningless. Correct: $\alpha(n) \leq 4$.

A simpler bound: $O(m \log^* n)$ (proof sketch)

$\log^* n$ = the number of times $\log_2$ must be applied to reach ≤ 1. ($\log^* 2^{65536} = 5$.)

Facts about ranks (with union by rank + compression): ranks only increase along a path to the root; a node of rank r has had $\geq 2^r$ descendants, so at most $n/2^r$ nodes ever have rank r. (The first fact is a warm-up; the second is the PS3 lemma.)

1. **Rank groups:** group g holds ranks in $(t_{g-1}, t_g]$, with $t_0 = 1,\ t_g = 2^{t_{g-1}}$ (towers of 2). There are $\leq \log^* n + 1$ groups.
2. Charge each step of a FIND from node x to its parent:
   - to the **FIND**, if the parent is in a different group or is the root: $\leq \log^* n + 2$ such steps per FIND;
   - otherwise to the **node** x. After compression x's parent has a strictly higher rank, so x is charged at most t_g times while in group g.
3. Nodes in group g: $\leq \sum_{r > t_{g-1}} n/2^r \leq n/2^{t_{g-1}} = n/t_g$. Total node charges per group $\leq n$, so $\leq n(\log^* n + 1)$ overall.

Total: $O\big((m+n)\log^* n\big)$ amortized.

Exercise B: DSU reasoning

1. Kruskal on edges that arrive pre-sorted: what is the total cost? Which term dominates for $m = 10^7$, $n = 10^6$?
2. **Rollback DSU:** offline dynamic connectivity needs to *undo* the most recent union. Why must path compression be disabled? What is the cost per operation with union by size and no compression?
3. **Latest free slot (L08):** accept unit jobs in decreasing penalty order and put each one in the latest free slot $\leq$ its deadline. Show how DSU finds that slot, and the total cost.
4. Can a single FIND take $\Theta(\log n)$ time with both heuristics? Can it take $\Theta(n)$?

Pairs · 8 min

BLOCK C · 02:10–02:55

Evidence and design

Density decides · DSU heuristics measured · MST variants

Experiment 1: MST algorithms by density

m	m/n	Kruskal	Prim (heap)	Prim (array)
11,959	3	**0.7**	1.2	42.0
84,174	21	**4.8**	8.6	43.2
803,489	201	53.8	110.8	**44.2**
7,998,000	2000	623.0	2250.9	**47.0**

ms; $n = 4000$, random connected graphs, 32-bit random weights; all three agree on the MST weight. `Student_Resources/mst_dsu_bench.cpp`, g++ 15.2 -O2.

INTERPRET · 4 MIN

1. Predict the crossover density between array-Prim and the others from the bounds. Where is it measured?

2. Why is array-Prim's time almost constant across rows?

3. Kruskal beats heap-Prim everywhere here. Is that guaranteed?

Experiment 2: DSU heuristics

2a. Adversarial path (unite $i, i+1$ for all i, then n finds of element 0): pointer steps per find.

n	naive	size only	compress only	size + compress
10,000	9,999	1.0	2.0	1.0
160,000	159,999	1.0	2.0	1.0

2b. Random workload (n random unions interleaved with $4n$ random finds): pointer steps per operation.

n	naive	size only	compress only	size + compress
10^5	1,324	1.08	1.76	0.70
10^6	(too slow)	1.09	2.03	0.70

INTERPRET · 4 MIN

1. Why is naive linking catastrophic on 2a but the others are not?

2. What do random workloads *fail* to show about the heuristics?

3. What would you have to construct to see $\log n$ growth for "size only"?

Exercise C: MST variants (design + proof)

1. **k-clustering with maximum spacing.** Partition n points into k non-empty clusters to maximise the minimum distance between points in different clusters. Show that "run Kruskal and stop when k components remain" is optimal.
2. **Incremental MST.** Given an MST T, a new edge $e = (u, v)$ is added to G. Update the MST in $O(n)$ time, and prove your update is correct.
3. **Second-best MST.** Find a spanning tree of minimum weight among those different from the MST (assume distinct weights). Give an $O(n^2)$ algorithm, assuming the MST is given.

Groups · 12 min

Synthesis

1. **Cut property** (safe edges) and **cycle property** (unsafe edges): every MST algorithm applies them in some order.
2. Kruskal: sort + DSU. Prim: the priority queue decides: array n^2, binary heap $m \log n$, Fibonacci heap $m + n \log n$. Borůvka: parallel phases, needs consistent tie-breaking.
3. Choose by density, by the representation (are edges stored or computed?), and by the environment (distributed? pre-sorted?).
4. DSU: size/rank ⇒ $O(\log n)$ worst case; + compression ⇒ $O(m\,\alpha(n))$ **amortized**, and tight.
5. Random workloads do not reveal worst-case behaviour. Construct adversaries to test guarantees.

NEXT WEEK · L10 (SAT 28 NOV)

Shortest paths organised by **assumptions**: unweighted, DAG, non-negative, negative edges, negative cycles, single-source vs. all-pairs, sparse vs. dense, and potentials / Johnson's algorithm.

Read: CLRS 4e Ch. 22–23. **Project M4** (working implementation) due Sat 28 Nov. **PS3** due Sat 05 Dec.